

TARUN BEHERA
202501090092
ME2 M21

1.1.1. Calculate Momentum

06:58

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: where: is the mass of the object (in kilograms). is the velocity of the object (in meters per second).

Input

Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places)



Sample Test Cases

Test case 1

5.0

10.0

50.00kgm/s

Test case 2

2.3

4.8

11.04kgm/s

```
m=float(input()) v=float(input())  
p=m*v  
print(f"{p:.2f}kgm/s")
```

Average time
0.004 s
3.50 ms

Maximum time
0.005 s
5.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 4 ms Debug

Expected output	Actual output
5.0	5.0
10.0	10.0
50.00kgm/s	50.00kgm/s

Test case 2 3 ms

Terminal Test cases

1.1.2. Conditional Calculation Based on the Number of Digits

01:40

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraint

s: $1 \leq \leq$
999

Input

Format:

- The input consists of a single integer .

Output

Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".



Sample Test Cases

Test case 1

9

81

Test case 2

166

5.50

Test case 3

24

4.90

Test case 4

3456

Invalid

```
n=int(input()) if 1<=n<=9:
```

```
    print(n*n)
```

```
elif 10<=n<=99:
```

```
    print(f"{n**(1/2):.2f}") elif
```

```
99<=n<=999:
```

```
    print(f"{n**(1/3):.2f}")
```

```
else:
```

```
    print("Invalid")
```

The screenshot shows a code execution environment with a summary bar at the top indicating '4 out of 4 shown test case(s) passed' and '3 out of 3 hidden test case(s) passed'. Below this, a table displays the results for three test cases. Test case 1 shows an expected output of 9 and an actual output of 9. Test case 2 and Test case 3 are also shown as passed. At the bottom, there are tabs for 'Terminal' and 'Test cases'.

Test Case	Expected output	Actual output
Test case 1	9	9
Test case 2		
Test case 3		

1.1.3Write a Python program to calculate number of days between two dates.

Input

Format:

- The first line contains the first date in the format .
- The second line contains the second date in the format .

Output Format:

- An integer representing the number of days between the given dates.

```
from datetime import date
```

**Not
e:**

- The first date should always be considered earlier than the second date.



Sample Test Cases

Test case 1

2014-07-02

2014-11-02

123 ↵

Test case 2

2014-07-02

2014-07-11

9 ↵

```
date_str1 = input() date_str2
```

```
= input()
```

```
y1, m1, d1 = map(int, date_str1.split('-')) y2, m2,
```

```
d2 = map(int, date_str2.split('-'))
```

```
first_date = date(y1, m1, d1) second_date
```

```
= date(y2, m2, d2)
```

```
delta = second_date - first_date
```

```
print(delta.days)
```

```
n=int(input()) reverse_number =
```

Average time
0.003 s
3.25 ms

Maximum time
0.006 s
6.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 6 ms
Debug

Expected output	Actual output
2014-07-02	2014-07-02
2014-11-02	2014-11-02
123	123

✓ Test case 2 3 ms

1.1.4 Write a Python program to reverse the digits of the number and print the reversed number.

Input

Format:

- The input is a single integer.

Output

Format:

- The output prints a single integer, which is the reversed number.



Sample Test Cases

Test case 1

5367

7635 ↵

Test case 2

0132

231 ↵

```
int(str(n)[::-1])
```

```
print(reverse_number)
```

Average time
0.002 s
1.80 ms

Maximum time
0.003 s
3.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 2 ms
Debug

Expected output
Actual output

5367
5367

7635
7635

✓ Test case 2 2 ms

Terminal
Test cases

1.1.5 Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

```
<number> x <multiplier> = <result>
>
```

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.



Sample Test Cases

Test case 1

8

8 • x • 1 • = • 8 ↵

8 • x • 2 • = • 16 ↵

8 • x • 3 • = • 24 ↵

8 • x • 4 • = • 32 ↵

8 • x • 5 • = • 40 ↵

8 • x • 6 • = • 48 ↵

8 • x • 7 • = • 56 ↵

8 • x • 8 • = • 64 ↵

8 • x • 9 • = • 72 ↵

8 · x · 10 · = · 80 ↵

Test case 2

5

5 · x · 1 · = · 5 ↵

5 · x · 2 · = · 10 ↵

5 · x · 3 · = · 15 ↵

5 · x · 4 · = · 20 ↵

5 · x · 5 · = · 25 ↵

5 · x · 6 · = · 30 ↵

5 · x · 7 · = · 35 ↵

5 · x · 8 · = · 40 ↵

5 · x · 9 · = · 45 ↵

5 · x · 10 · = · 50 ↵

```
n = int(input()) for i in range(1,11):
```

```
    print(f"{n} x {i}
```

```
    = {n*i}")
```

Average time

0.003 s

3.25 ms

Maximum time

0.005 s

5.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1

5 ms

Debug

≡

📄

Expected output

8

8 · x · 1 · = · 8

8 · x · 2 · = · 16

8 · x · 3 · = · 24

8 · x · 4 · = · 32

8 · x · 5 · = · 40

Actual output

8

8 · x · 1 · = · 8

8 · x · 2 · = · 16

8 · x · 3 · = · 24

8 · x · 4 · = · 32

8 · x · 5 · = · 40

1.2.1 Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- ☐ If the aggregate percentage is greater than 75, the grade is Distinction.
- ☐ If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- ☐ If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- ☐ If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- ☐ The first input will be an integer , the number of courses.

- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Not e:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.



Sample Test Cases

Test case 1

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 2

5

56 78 97 86 93

Aggregate Percentage: 82.00

Grade: Distinction

Test case 3

5

99 85 43 97 34

Fail

Test case 4

3

55 54 53

Aggregate Percentage: 54.00

Grade: Second Division

Test case 5

5

40 45 42 48 41

Aggregate Percentage: 43.20

Grade: Third Division

```
def calculate_grade(marks):
```

```
    if any(mark < 40 for mark in marks):
```

```

        return "Fail"

    percentage = sum(marks)/len(marks)

# print(percentage)    if percentage>75:

        grade = "Distinction"

    elif percentage>=60:

        grade = "First Division"

elif percentage>=50:

        grade = "Second Division"

elif percentage>=40:

        grade = "Third Division"

    else:

        grade = "Fail"

# print(grade)  output = f"Aggregate Percentage:
{percentage:.2f}\nGrade: {grade}"

    # return f"Aggregate Percentage: {percentage:.2f}\nGrade:{grade}"

    return output

n = int(input()) marks =
map(int,input().split(" ")) marks = list(marks)
# print(marks)
print(calculate_grade(marks))

```

Average time: 0.003 s 2.90 ms	Maximum time: 0.005 s 5.00 ms	5 out of 5 shown test case(s) passed 5 out of 5 hidden test case(s) passed
--	--	---

Test case 1	Expected output	Actual output
4 ms	4	4
55 65 78 89	55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First Division	Grade: First Division	Grade: First Division

Test case 2	4 ms
-------------	------

1.2.2 Write a Python program that uses recursion to print the terms of the first Fibonacci series.

Input Format:

□ A single integer representing the number of terms to generate.

Output Format:

□ A single line containing the first terms of the Fibonacci sequence, separated by spaces.



Sample Test Cases

Test case 1

5

0 • 1 • 1 • 2 • 3 •

Test case 2

15

0 • 1 • 1 • 2 • 3 • 5 • 8 • 13 • 21 • 34 • 55 • 89 • 144 • 233 • 377 •

```
def fibonacci(n):  
    if n == 1:  
        return 0  
    elif n == 2:  
        return 1  
    else:  
        # write the code..  
  
        return fibonacci(n - 1) + fibonacci(n-2)  
  
n = int(input())  
for i in range(1,  
n + 1):  
    print(fibonacci(i), end=" ")
```

Average time
0.007 s
7.25 ms

Maximum time
0.018 s
18.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 3 ms Debug

Expected output

Actual output

5

5

0 1 1 2 3

0 1 1 2 3

Test case 2 1 ms

1.2.3 Write a Python program to print a pattern of asterisks in the form of a rightangled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Not Refer to the displayed test cases for the sample pattern.

e:

Sample Test Cases

Test case 1

5

*.
..
..*.
..*.*.
..*.*.*.

Test case 2

4

*.
..
..*.
..*.*.

```
n = int(input())
for i in range (1,1+n):
    print('* ' * i)
```

Average time

0.003 s

2.83 ms



Maximum time

0.006 s

6.00 ms



✓ 2 out of 2 shown test case(s) passed

✓ 4 out of 4 hidden test case(s) passed

✓ Test case 1 2 ms

Debug



Expected output

5
*
* *
* * *
* * * *
* * * * *

Actual output

5
*
* *
* * *
* * * *
* * * * *

1.2.4 Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed testn=int(input())
- for i in range(1,n+1):
- for j in range(1,i+1):
- print(j,end=" ")
- print("") cases for the sample pattern.



Sample Test Cases

Test case 1

5

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

Test case 2

8

1

```

1•2•
1•2•3•
1•2•3•4•
1•2•3•4•5•
1•2•3•4•5•6•
1•2•3•4•5•6•7•
1•2•3•4•5•6•7•8•

```

```

n=int(input())
for i in range(1,n+1):
    for j in range(1,i+1):
        print(j,end=" ")
    print("")

```

Average time
0.003 s
3.25 ms

Maximum time
0.005 s
5.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 5 ms Debug

Expected output	Actual output
5	5
1•	1•
1•2•	1•2•
1•2•3•	1•2•3•
1•2•3•4•	1•2•3•4•
1•2•3•4•5•	1•2•3•4•5•